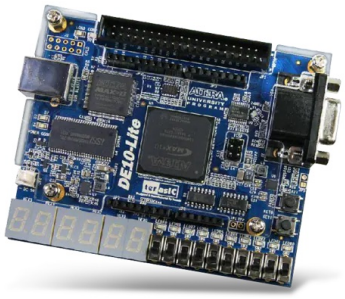
instituto superior de engenharia de lisboa

Licenciatura em Engenharia Informática e de Computadores

Lógica e Sistemas Digitais
2026 / 2027 inverno



Relatório do Trabalho Laboratorial 1
Sistema de Controlo de Acessos

Trabalho realizado por:

Número: 56012 Nome: João Lucas Passos Ferreira

Número: 56010 Nome: André Vinagre Granja

Turma: 11D
Docente: Pedro Miguens Matutino

Índice

Lista de Figuras ii

Lista de Tabelas ii

Lista de VHDL ii

1 Objetivo 1

2 Descrição do Circuito a Projetar 1

3 Desenvolvimento do Projeto 1
3.1 Diagrama de blocos 2
3.2 Funções lógicas dos circuitos 2
3.3 Simplificação lógica 3
3.4 Implementação em VHDL 4

4 Resultados e Discussão 5
4.1 Simulação 5
4.2 Teste 6

5 Conclusão 6

Referências 8

A VHDL 9

B Atribuição de Pinos 16

Lista de Figuras

Figura 1 Diagrama de blocos do sistema de controlo de acessos. 2
Figura 2 Mapa de Karnaugh de D_0 e respetivos agrupamentos. 3
Figura 3 Mapa de Karnaugh de D_1 e respetivos agrupamentos. 4
Figura 4 Mapa de Karnaugh de D_2 e respetivos agrupamentos (o agrupamento azul envolve as colunas extremas, vizinhas na topologia toroidal do mapa). 4
Figura 5 Mapa de Karnaugh de D_3 e respetivos agrupamentos. 5

Lista de Tabelas

Tabela 1 Tabela de verdade global do sistema de controlo de acessos. . 2
Tabela 2 Resultados da simulação dos três blocos (ModelSim-Altera 20.1). 6
Tabela 3 Atribuição de pinos do módulo `Controlo_Acessos` na placa Intel DE10-Lite [2]. 16

Lista de VHDL

VHDL 1 Bloco 1 — `Identifica_Departamento`: descodificação do código do funcionário nas permissões dos departamentos 9
VHDL 2 Bloco 2 — `Verifica`: validação do departamento ativo contra as permissões 9
VHDL 3 Módulo de topo — `Controlo_Acessos`: interligação estrutural dos dois blocos 10
VHDL 4 Testbench de `Identifica_Departamento` 11
VHDL 5 Testbench de `Verifica` 12
VHDL 6 Testbench de `Controlo_Acessos` 13

1 Objetivo

O objetivo do Trabalho Laboratorial 1 é projetar, implementar e verificar um circuito digital combinatório que realize um **sistema de controlo de acessos** a um edifício com quatro departamentos. O circuito recebe o código de um funcionário e o departamento onde este se pretende identificar, e decide automaticamente se o acesso é autorizado.

O projeto foi desenvolvido segundo o fluxo *especificação* → *simplificação lógica* → *descrição em VHDL* → *simulação* → *síntese e programação*, recorrendo às ferramentas *ModelSim-Altera 20.1* para simulação e *Quartus Prime Lite 20.1* para síntese, *fitting* e programação da placa **Intel DE10-Lite** (dispositivo MAX 10, `10M50DAF484C7G`).

2 Descrição do Circuito a Projetar

Cada funcionário é identificado por um **código binário de 4 bits** $A = A_3A_2A_1A_0$, representando os funcionários de 0 a 15. A cada código corresponde um conjunto de *permissões de acesso*, ou seja, os departamentos $\{D_0, D_1, D_2, D_3\}$ onde esse funcionário pode entrar. A cada departamento associa-se um bit de permissão: $D = D_3D_2D_1D_0$ (`1` = autorizado, `0` = não autorizado).

O sistema dispõe ainda de uma entrada de **seleção de departamento** $S = S_1S_0$ (`00` = D_0, `01` = D_1, `10` = D_2, `11` = D_3). A saída R controla a cancela: $R = 1$ *abre* a cancela (acesso concedido) quando o código do funcionário tem permissão para o departamento selecionado; caso contrário, $R = 0$ mantém a cancela *fechada*.

O comportamento pretendido é descrito pela tabela de verdade global da Tabela 1, que constitui a especificação de referência do projeto (para todos os 16 códigos de funcionário).

3 Desenvolvimento do Projeto

O sistema foi decomposto em dois blocos combinatórios independentes, interligados por um módulo de topo:

1. `Identifica_Departamento` — descodifica o código do funcionário A no vetor de permissões D;
2. `Verifica` — verifica se o departamento selecionado S está autorizado em D e gera o sinal de controlo da cancela R;
3. `Controlo_Acessos` — módulo de topo que instancia e liga os dois blocos anteriores.

Tabela 1: Tabela de verdade global do sistema de controlo de acessos.

Func.	A_3	A_2	A_1	A_0	D_3	D_2	D_1	D_0	**Departamentos**
0	0	0	0	0	0	0	0	**1**	D_0
1	0	0	0	1	0	**1**	0	0	D_2
2	0	0	1	0	0	0	**1**	0	D_1
3	0	0	1	1	**1**	0	0	0	D_3
4	0	1	0	0	0	**1**	**1**	0	D_1, D_2
5	0	1	0	1	0	**1**	0	**1**	D_2, D_0
6	0	1	1	0	0	0	0	**1**	D_0
7	0	1	1	1	0	**1**	0	0	D_2
8	1	0	0	0	**1**	0	0	0	D_3
9	1	0	0	1	**1**	0	0	0	D_3
10	1	0	1	0	0	0	0	**1**	D_0
11	1	0	1	1	0	**1**	0	**1**	D_2, D_0
12	1	1	0	0	0	**1**	0	**1**	D_2, D_0
13	1	1	0	1	0	0	**1**	0	D_1
14	1	1	1	0	0	**1**	**1**	0	D_2, D_1
15	1	1	1	1	**1**	0	**1**	0	D_1, D_3

3.1 Diagrama de blocos

A arquitetura do sistema encontra-se representada na Figura 1. O bloco `Identifica_Departamento` recebe o código do funcionário $A[3..0]$ e produz as permissões $D[3..0]$. O bloco `Verifica` recebe essas permissões, juntamente com a seleção de departamento $S[1..0]$, e gera a saída R.

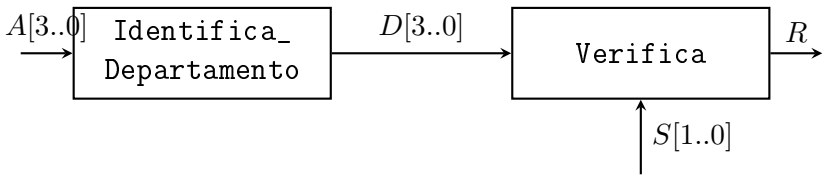


Figura 1: Diagrama de blocos do sistema de controlo de acessos.

3.2 Funções lógicas dos circuitos

A partir da Tabela 1 obtêm-se diretamente as formas canónicas *soma-de-produtos* (SOP) de cada bit de permissão, identificando os mintermos em que a saída vale 1:

$$D_0(A_3, A_2, A_1, A_0) = \sum m(0, 5, 6, 10, 11, 12) \tag{1}$$

$$D_1(A_3, A_2, A_1, A_0) = \sum m(2, 4, 13, 14, 15) \tag{2}$$

$$D_2(A_3, A_2, A_1, A_0) = \sum m(1, 4, 5, 7, 11, 12, 14) \tag{3}$$

$$D_3(A_3, A_2, A_1, A_0) = \sum m(3, 8, 9, 15) \tag{4}$$

O bloco `Verifica` seleciona, de entre as quatro permissões D_i, aquela que corresponde ao departamento ativo S, funcionando como um multiplexador controlado por S:

$$R = D_0 \cdot \overline{S_1} \cdot \overline{S_0} + D_1 \cdot \overline{S_1} \cdot S_0 + D_2 \cdot S_1 \cdot \overline{S_0} + D_3 \cdot S_1 \cdot S_0 \tag{5}$$

3.3 Simplificação lógica

Cada função D_i foi simplificada com o auxílio do mapa de Karnaugh 4×4 (linhas em A_3A_2, colunas em A_1A_0, ambas na ordem de Gray $00, 01, 11, 10$). Os agrupamentos usados (a cor com que aparecem em cada mapa) estão assinalados sobre as células correspondentes.

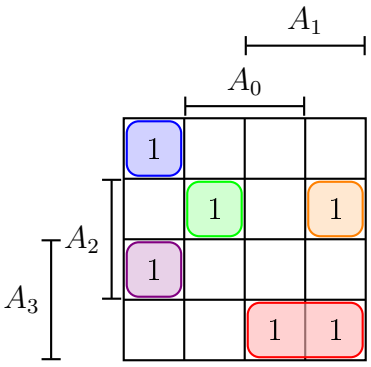


Figura 2: Mapa de Karnaugh de D_0 e respetivos agrupamentos.

Função D_0:

$$\begin{aligned} D_0 = {} & A_3 \cdot \overline{A_2} \cdot A_1 + \overline{A_3} \cdot A_2 \cdot \overline{A_1} \cdot A_0 + \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} \\ & + \overline{A_3} \cdot A_2 \cdot A_1 \cdot \overline{A_0} + A_3 \cdot A_2 \cdot \overline{A_1} \cdot \overline{A_0} \end{aligned} \tag{6}$$

Função D_1:

$$D_1 = A_3 \cdot A_2 \cdot A_0 + A_3 \cdot A_2 \cdot A_1 + \overline{A_3} \cdot \overline{A_2} \cdot A_1 \cdot \overline{A_0} + \overline{A_3} \cdot A_2 \cdot \overline{A_1} \cdot \overline{A_0} \tag{7}$$

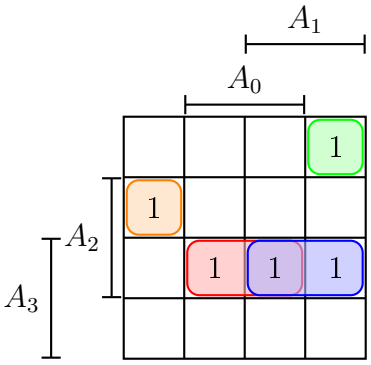


Figura 3: Mapa de Karnaugh de D_1 e respetivos agrupamentos.

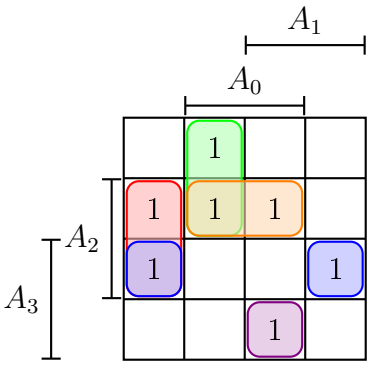


Figura 4: Mapa de Karnaugh de D_2 e respetivos agrupamentos (o agrupamento azul envolve as colunas extremas, vizinhas na topologia toroidal do mapa).

Função D_2:

$$\begin{aligned} D_2 = {} & A_2 \cdot \overline{A_1} \cdot \overline{A_0} + A_3 \cdot A_2 \cdot \overline{A_0} + \overline{A_3} \cdot \overline{A_1} \cdot A_0 \\ & + \overline{A_3} \cdot A_2 \cdot A_0 + A_3 \cdot \overline{A_2} \cdot A_1 \cdot A_0 \end{aligned} \tag{8}$$

Função D_3:

$$D_3 = A_3 \cdot \overline{A_2} \cdot \overline{A_1} + \overline{A_3} \cdot \overline{A_2} \cdot A_1 \cdot A_0 + A_3 \cdot A_2 \cdot A_1 \cdot A_0 \tag{9}$$

A função de verificação (5) já se encontra na forma mínima, dispensando simplificação adicional.

3.4 Implementação em VHDL

Os três blocos foram descritos em VHDL (biblioteca `ieee.std_logic_1164`), recorrendo exclusivamente a **atribuições concorrentes de sinal** com os operadores

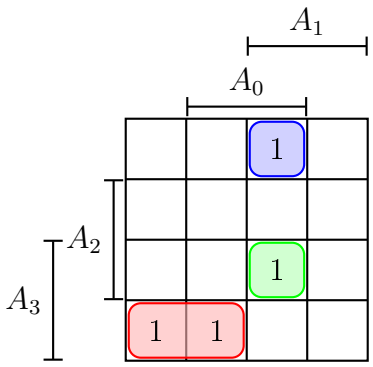


Figura 5: Mapa de Karnaugh de D_3 e respetivos agrupamentos.

lógicos `and`, `or` e `not` — consequentemente, o circuito é puramente combinatório e não contém processos nem memória.

O bloco `Identifica_Departamento` (VHDL 1) implementa diretamente as equações simplificadas de D_0 a D_3. O bloco `Verifica` (VHDL 2) implementa a expressão (5). O módulo de topo `Controlo_Acessos` (VHDL 3) instancia os dois blocos anteriores através de `entity work` e liga a saída de permissões do primeiro à entrada de permissões do segundo pelo sinal interno `internal_D`, conforme a Figura 1. A descrição [1] orientou a estrutura do projeto no Quartus.

4 Resultados e Discussão

A verificação do circuito foi feita em duas fases: (i) simulação funcional com *testbenches* em ModelSim-Altera 20.1; (ii) síntese e verificação de pinos no Quartus Prime Lite 20.1 para a placa DE10-Lite.

4.1 Simulação

Foram escritos *testbenches* VHDL para cada bloco, que estimulam todas as combinações de entrada relevantes e comparam as saídas com os valores esperados da especificação, emitindo mensagens de `assert` em caso de erro. Os testbenches `Identifica_Departamento_tb` (VHDL 4), `Verifica_tb` (VHDL 5) e `Controlo_Acessos_tb` (VHDL 6) foram executados em modo batch com `vsim -c`; os resultados obtidos resumem-se na Tabela 2.

O testbench de `Identifica_Departamento` percorre os 16 códigos de funcionário e confirma que o vetor de permissões D coincide linha a linha com a Tabela 1. O testbench de `Verifica` testa os quatro departamentos (incluindo casos de acesso concedido e negado) e o testbench do módulo de topo valida a integração dos dois

Tabela 2: Resultados da simulação dos três blocos (ModelSim-Altera 20.1).

Testbench	Entidade testada	Casos	Resultado
`Identifica_Departamento_tb`	`Identifica_Departamento`	16/16	PASS
`Verifica_tb`	`Verifica`	4/4	PASS
`Controlo_Acessos_tb`	`Controlo_Acessos`	5/5	PASS

blocos em cinco cenários representativos. Todas as asserções passaram, não se tendo detetado qualquer discrepância face à especificação.

4.2 Teste

O projeto foi sintetizado no Quartus Prime Lite 20.1 para o dispositivo MAX 10 `10M50DAF484C7G` da placa Intel DE10-Lite. A análise e síntese (`quartus_map`) terminou sem erros e o *fitter* (`quartus_fit`) terminou sem erros, com apenas seis avisos informativos da plataforma (AN 447 relativo ao standard de 3,3 V e *Missing drive strength*), que não comprometem o funcionamento.

As entradas A e S foram atribuídas a interruptores (`SW`) e a saída R a um LED (`LEDR0`), de acordo com a atribuição de pinos da Tabela 3 (Apêndice B). Os sinais usam o standard de E/S `3.3-V LVTTL`.

O teste na placa foi realizado num MacBook Pro com uma partição de Windows 10. Nessa partição foram instalados o Quartus Prime Lite e o controlador do USB-Blaster, permitindo programar a placa DE10-Lite. Na primeira tentativa, o comportamento observado não correspondia ao esperado: os pinos associados a A_2 e A_1 estavam trocados, fazendo com que os interruptores que representavam esses bits do código do funcionário fossem interpretados de forma incorreta. Para poder concluir o teste, a atribuição foi corrigida temporariamente no próprio Quartus; depois dessa alteração, a programação e o funcionamento da placa foram bem-sucedidos. Por fim, a correção foi aplicada definitivamente no código e na atribuição de pinos do projeto, ficando o mapeamento final apresentado na Tabela 3.

5 Conclusão

O trabalho consistiu em projetar, descrever em VHDL e verificar um sistema digital combinatório de controlo de acessos a um edifício com quatro departamentos. O sistema foi desenvolvido a partir de uma tabela de verdade global, decomposto em dois blocos (`Identifica_Departamento` e `Verifica`) interligados por um módulo de topo (`Controlo_Acessos`), com as funções lógicas simplificadas por mapas

de Karnaugh.
A implementação foi simulada com sucesso em ModelSim-Altera 20.1 (todas as asserções dos testbenches passaram) e sintetizada sem erros no Quartus Prime Lite 20.1 para a placa Intel DE10-Lite, onde foi definida a atribuição de pinos aos interruptores e ao LED.
O teste experimental foi realizado através de um MacBook Pro com Windows 10, utilizando o Quartus Prime Lite e o USB-Blaster para programar a placa. Na primeira tentativa foram detetados os pinos A_2 e A_1 trocados, o que fazia com que os códigos dos funcionários fossem interpretados incorretamente. A atribuição foi corrigida temporariamente no Quartus, permitindo confirmar o funcionamento do circuito, e posteriormente a correção foi aplicada definitivamente no código e no projeto. O mapeamento final é o apresentado no Apêndice B. O código-fonte e os ficheiros do projeto estão disponíveis no repositório GitHub indicado em [3].

Referências

[1] Pedro Miguens Matutino, Sérgio André *Guia de Instalação do Quartus Prime Lite Edition 20.1.1.720*, 2022, ISEL.

[2] Terasic *DE10-lite User Manual*, 2020, Terasic.

[3] Repositório GitHub do projeto, `https://github.com/joaolucaspfer/TLAB1-LSD-ISEL`.

A VHDL

VHDL 1: Bloco 1 — `Identifica_Departamento`: descodificação do código do funcionário nas permissões dos departamentos

```vhdl
-- ================================================================================
-- Ficheiro    : Identifica_Departamento.vhd
-- Bloco       : 1 -- Controlo de Acessos (TLAB1)
-- Especifica.: sdd/specs/01_identifica_departamento.md (REQ-ID-001..008, DEC-001)
-- Descricao   : Combinacional que decodifica o codigo do funcionario (A, 4 bits,
--               0..15) no vetor de permissoes dos departamentos (D, 4 bits).
-- Biblioteca : ieee.std_logic_1164.all  (REQ-ID-001)
-- ================================================================================

library ieee;
use ieee.std_logic_1164.all;

-- ================================================================================
-- Entidade Identifica_Departamento (REQ-ID-002)
--   Entrada A (4 bits): codigo do funcionario, A(3)=A3 ... A(0)=A0.
--   Saida   D (4 bits): permissoes dos departamentos, D(3)=D3 ... D(0)=D0 (1=autorizado).
-- ================================================================================
entity Identifica_Departamento is
    port (
        A : in  std_logic_vector(3 downto 0);
        D : out std_logic_vector(3 downto 0)
    );
end entity Identifica_Departamento;

-- ================================================================================
-- Arquitetura Behavioral (REQ-ID-003): atribuicoes concorrentes de sinal, sem processos.
-- Cada linha abaixo e uma atribuicao concorrente (equacao booleana das seccoes 4/5 da
-- spec), construida apenas com os operadores logicos and, or e not.
-- ================================================================================
architecture Behavioral of Identifica_Departamento is
begin

    -- --------------------------------------------------------------------------
    -- D(0) -- REQ-ID-005 | Mintermos: m0, m5, m6, m10, m11, m12
    -- --------------------------------------------------------------------------
    D(0) <= (A(3) and not A(2) and A(1))                          -- m10, m11
         or (not A(3) and A(2) and not A(1) and A(0))             -- m5   [DEC-001]
         or (not A(3) and not A(2) and not A(1) and not A(0))     -- m0
         or (not A(3) and A(2) and A(1) and not A(0))             -- m6
         or (A(3) and A(2) and not A(1) and not A(0));            -- m12

    -- --------------------------------------------------------------------------
    -- D(1) -- REQ-ID-006 | Mintermos: m2, m4, m13, m14, m15
    -- --------------------------------------------------------------------------
    D(1) <= (A(3) and A(2) and A(0))                              -- m13, m15
         or (A(3) and A(2) and A(1))                              -- m14, m15
         or (not A(3) and not A(2) and A(1) and not A(0))         -- m2
         or (not A(3) and A(2) and not A(1) and not A(0));        -- m4

    -- --------------------------------------------------------------------------
    -- D(2) -- REQ-ID-007 | Mintermos: m1, m4, m5, m7, m11, m12, m14
    -- --------------------------------------------------------------------------
    D(2) <= (A(2) and not A(1) and not A(0))                      -- m4, m12
         or (A(3) and A(2) and not A(0))                          -- m12, m14
         or (not A(3) and not A(1) and A(0))                      -- m1, m5
         or (not A(3) and A(2) and A(0))                          -- m5, m7
         or (A(3) and not A(2) and A(1) and A(0));                -- m11

    -- --------------------------------------------------------------------------
    -- D(3) -- REQ-ID-008 | Mintermos: m3, m8, m9, m15
    -- --------------------------------------------------------------------------
    D(3) <= (A(3) and not A(2) and not A(1))                      -- m8, m9
         or (not A(3) and not A(2) and A(1) and A(0))             -- m3
         or (A(3) and A(2) and A(1) and A(0));                    -- m15

end architecture Behavioral;
```

VHDL 2: Bloco 2 — `Verifica`: validação do departamento ativo contra as permissões

```vhdl
-- ------------------------------------------------------------------------
-- Verifica.vhd
-- Segundo bloco do sistema: verifica se o departamento ativo (S) esta
-- autorizado (D) e gera o sinal de controlo da cancela (R).
```

```
-- ------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

-- ------------------------------------------------------------------------------
-- Entidade
--    D : 4 bits - departamentos autorizados (D(0)..D(3))
--    S : 2 bits - departamento ativo no sistema
--          00 = D0, 01 = D1, 10 = D2, 11 = D3
--    R : 1 bit  - sinal de controlo da cancela (1 = Abre, 0 = Fechada)
-- ------------------------------------------------------------------------------
entity Verifica is
    port (
        D : in  std_logic_vector(3 downto 0);
        S : in  std_logic_vector(1 downto 0);
        R : out std_logic
    );
end entity Verifica;

-- ------------------------------------------------------------------------------
-- Arquitetura Behavioral
--    R = (D(0) AND NOT S(1) AND NOT S(0))  -- S = 00 -> D0
--      OR (D(1) AND NOT S(1) AND     S(0))  -- S = 01 -> D1
--      OR (D(2) AND     S(1) AND NOT S(0))  -- S = 10 -> D2
--      OR (D(3) AND     S(1) AND     S(0))  -- S = 11 -> D3
-- ------------------------------------------------------------------------------
architecture Behavioral of Verifica is
begin

    R <= (D(0) and (not S(1)) and (not S(0))) or
         (D(1) and (not S(1)) and      S(0) ) or
         (D(2) and      S(1)  and (not S(0))) or
         (D(3) and      S(1)  and      S(0) );

end architecture Behavioral;
```

VHDL 3: Módulo de topo — `Controlo_Acessos`: interligação estrutural dos dois blocos

```
-- ------------------------------------------------------------------------------
-- Controlo_Acessos.vhd
-- Modulo principal: interliga os blocos Identifica_Departamento e Verifica.
--    A (codigo do funcionario) -> Identifica_Departamento -> internal_D
--    internal_D + S (departamento ativo) -> Verifica -> R (cancela)
-- Conforme SPEC-CA-05.
-- ------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

-- ------------------------------------------------------------------------------
-- Entidade
--    A : 4 bits - codigo do funcionario (0..15)
--    S : 2 bits - departamento ativo (00=D0, 01=D1, 10=D2, 11=D3)
--    R : 1 bit  - sinal de controlo da cancela (1 = Abre, 0 = Fechada)
-- ------------------------------------------------------------------------------
entity Controlo_Acessos is
    port (
        A : in  std_logic_vector(3 downto 0);
        S : in  std_logic_vector(1 downto 0);
        R : out std_logic
    );
end entity Controlo_Acessos;

-- ------------------------------------------------------------------------------
-- Arquitetura Behavioral (estrutural): instancia os dois blocos existentes e
-- liga a saida de permissoes do primeiro a' entrada de permissoes do segundo
-- atraves do sinal interno internal_D.
-- ------------------------------------------------------------------------------
architecture Behavioral of Controlo_Acessos is

    -- Liga a saida D do Identifica_Departamento a' entrada D do Verifica
    signal internal_D : std_logic_vector(3 downto 0);

begin

    -- Bloco 1: decodifica o codigo do funcionario nas permissoes D3..D0
    U_IDENTIFICA : entity work.Identifica_Departamento
        port map (
            A => A,
            D => internal_D
```

```
        );

    -- Bloco 2: valida o departamento ativo S contra as permissoes internal_D
    U_VERIFICA : entity work.Verifica
        port map (
            D => internal_D,
            S => S,
            R => R
        );

end architecture Behavioral;
```

VHDL 4: Testbench de `Identifica_Departamento`

```
-- ============================================================================================
-- Ficheiro    : Identifica_Departamento_tb.vhd
-- Bloco       : 1 -- Controlo de Acessos (TLAB1) -- TESTBENCH
-- Especifica.: sdd/specs/01_identifica_departamento.md (REQ-ID-009 / DEC-002 / TC-003)
-- Descricao   : Testbench sem portas do bloco Identifica_Departamento. O processo de
--               estimulos (stim_perc) percorre ciclicamente as 16 combinacoes de
--               entrada -- funcionario 0 ("0000") ate funcionario 15 ("1111") -- com
--               um atraso de 20 ns entre cada variacao, e compara a saida D com a
--               tabela de verdade global (sdd/specs/00_projeto_controlo_acessos.md).
--               Apos o 16.o estimulo termina com wait; (paragem da simulacao).
-- Ferramentas: ModelSim / QuestaSim (analise: vcom; simulacao: vsim -c).
-- ============================================================================================

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;  -- apenas para os relatorios (integer'image / to_integer)

-- ============================================================================================
-- Entidade do testbench -- sem portas (req. 1)
-- ============================================================================================
entity Identifica_Departamento_tb is
end;

architecture sim of Identifica_Departamento_tb is

    -- Sinais ligados a UUT
    signal A : std_logic_vector(3 downto 0) := (others => '0');
    signal D : std_logic_vector(3 downto 0);

    -- Atraso aplicado entre cada variacao de A (req. 3)
    constant TEMPO_ESTIMULO : time := 20 ns;

    -- Tabela de verdade de referencia (spec do projeto, sec. 4):
    --   ESPERADO(funcionario) = D3 & D2 & D1 & D0
    type t_permissoes is array (0 to 15) of std_logic_vector(3 downto 0);
    constant ESPERADO : t_permissoes := (
        "0001",  -- funcionario  0 ("0000"): acesso a D0
        "0100",  -- funcionario  1 ("0001"): acesso a D2
        "0010",  -- funcionario  2 ("0010"): acesso a D1
        "1000",  -- funcionario  3 ("0011"): acesso a D3
        "0110",  -- funcionario  4 ("0100"): acesso a D2, D1
        "0101",  -- funcionario  5 ("0101"): acesso a D2, D0
        "0001",  -- funcionario  6 ("0110"): acesso a D0
        "0100",  -- funcionario  7 ("0111"): acesso a D2
        "1000",  -- funcionario  8 ("1000"): acesso a D3
        "1000",  -- funcionario  9 ("1001"): acesso a D3
        "0001",  -- funcionario 10 ("1010"): acesso a D0
        "0101",  -- funcionario 11 ("1011"): acesso a D2, D0
        "0101",  -- funcionario 12 ("1100"): acesso a D2, D0
        "0010",  -- funcionario 13 ("1101"): acesso a D1
        "0110",  -- funcionario 14 ("1110"): acesso a D2, D1
        "1010"   -- funcionario 15 ("1111"): acesso a D3, D1
    );

begin

    -- ============================================================================================
    -- Unidade sob teste (req. 2)
    -- ============================================================================================
    UUT : entity work.Identifica_Departamento
        port map (
            A => A,
            D => D
        );

    -- ============================================================================================
    -- Processo de estimulos (req. 3): percorre ciclicamente as 16 combinacoes de
```

```
    -- entrada, de 20 ns em 20 ns, ate ao funcionario 15; termina com wait; (req. 5).
    -- ============================================================================
    stim_proc : process
        variable erros : natural := 0;
    begin
        report "=== Inicio da simulacao do Identifica_Departamento ===" severity note;

        for funcionario in 0 to 15 loop

            -- Aplica o codigo binario do funcionario atual a A:
            --   funcionario 0 -> "0000", funcionario 1 -> "0001", ...,
            --   funcionario 15 -> "1111"
            A <= std_logic_vector(to_unsigned(funcionario, A'length));

            -- Atraso de 20 ns: mantem o estimulo aplicado e deixa a saida estabilizar
            wait for TEMPO_ESTIMULO;

            -- Registo do funcionario em teste e dos valores observados
            report "Funcionario " & integer'image(funcionario)
                 & " | A = " & integer'image(to_integer(unsigned(A)))
                 & " | D = " & integer'image(to_integer(unsigned(D)))
                 & " | esperado = "
                 & integer'image(to_integer(unsigned(ESPERADO(funcionario))))
                severity note;

            -- Verificacao contra a tabela de verdade (TC-003 / REQ-ID-009)
            if D /= ESPERADO(funcionario) then
                erros := erros + 1;
                report "FALHA no funcionario " & integer'image(funcionario)
                     & " (codigo " & integer'image(to_integer(unsigned(A)))
                     & "): D = " & integer'image(to_integer(unsigned(D)))
                     & ", esperado " & integer'image(to_integer(unsigned(ESPERADO(funcionario))))
                    severity error;
            end if;

        end loop;

        -- Resumo final da simulacao
        if erros = 0 then
            report "=== SIMULACAO OK: 16/16 funcionarios corretos, 0 erros ==="
                severity note;
        else
            report "=== SIMULACAO COM ERROS: " & integer'image(erros)
                 & " divergencias ===" severity failure;
        end if;

        -- Paragem da simulacao apos o ciclo completo (req. 5)
        wait;

    end process stim_proc;

end architecture sim;
```

VHDL 5: Testbench de `Verifica`

```
-- ------------------------------------------------------------------------------
-- Verifica_tb.vhd
-- Testbench auto-verificavel do bloco Verifica (simulacao isolada).
-- Cenarios: D = "0101" (autorizado em D0 e D2), varrimento de S = 00..11
-- com 20 ns por estimulo, conforme SPEC-VTB-04.
-- ------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

-- ------------------------------------------------------------------------------
-- Entidade de testbench sem portas (padrao do projeto)
-- ------------------------------------------------------------------------------
entity Verifica_tb is
end entity Verifica_tb;

architecture sim of Verifica_tb is

    -- Sinais ligados a UUT
    signal D : std_logic_vector(3 downto 0);
    signal S : std_logic_vector(1 downto 0);
    signal R : std_logic;

    -- 20 ns por estimulo (REQ-VTB-003)
    constant TEMPO_ESTIMULO : time := 20 ns;

```

```
begin

    -- ------------------------------------------------------------------------
    -- Instanciacao da DUT
    -- ------------------------------------------------------------------------
    UUT : entity work.Verifica
        port map (
            D => D,
            S => S,
            R => R
        );

    -- ------------------------------------------------------------------------
    -- Processo de estimulos: 4 cenarios com D = "0101"
    -- ------------------------------------------------------------------------
    stim_proc : process
        variable erros : natural := 0;

        -- Compara R com o valor esperado e regista divergencias
        procedure verificar (esperado : std_logic; descricao : string) is
        begin
            if R /= esperado then
                erros := erros + 1;
                report descricao & " | ERRO: R=" & std_logic'image(R) &
                       ", esperado=" & std_logic'image(esperado)
                    severity error;
            else
                report descricao & " | OK: R=" & std_logic'image(R);
            end if;
        end procedure;

    begin
        -- Departamentos autorizados: D0 e D2 (D = "0101", D3..D0)
        D <= "0101";

        -- Teste 1: S = "00" -> departamento D0 ativo; D0 autorizado => R = 1 (Abre)
        S <= "00";
        wait for TEMPO_ESTIMULO;
        verificar('1', "Teste 1 | D=0101 | S=00 (D0 autorizado)");

        -- Teste 2: S = "01" -> departamento D1 ativo; D1 NAO autorizado => R = 0 (Fechada)
        S <= "01";
        wait for TEMPO_ESTIMULO;
        verificar('0', "Teste 2 | D=0101 | S=01 (D1 nao autorizado)");

        -- Teste 3: S = "10" -> departamento D2 ativo; D2 autorizado => R = 1 (Abre)
        S <= "10";
        wait for TEMPO_ESTIMULO;
        verificar('1', "Teste 3 | D=0101 | S=10 (D2 autorizado)");

        -- Teste 4: S = "11" -> departamento D3 ativo; D3 NAO autorizado => R = 0 (Fechada)
        S <= "11";
        wait for TEMPO_ESTIMULO;
        verificar('0', "Teste 4 | D=0101 | S=11 (D3 nao autorizado)");

        -- Resumo final (auto-verificacao, REQ-VTB-005)
        if erros = 0 then
            report "=== SIMULACAO OK: 4/4 testes corretos, 0 erros ===";
        else
            report "=== SIMULACAO FALHOU: " & natural'image(erros) &
                   " erro(s) em 4 testes ===" severity failure;
        end if;

        wait;  -- termina a simulacao (run -all)
    end process stim_proc;

end architecture sim;
```

VHDL 6: Testbench de `Controlo_Acessos`

```
-- ------------------------------------------------------------------------
-- Controlo_Acessos_tb.vhd
-- Testbench auto-verificavel do sistema global Controlo_Acessos.
-- Simula cenarios reais de acesso (funcionario x departamento ativo),
-- conforme SPEC-CTB-06.
-- ------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

-- ------------------------------------------------------------------------
```

```
12 -- Entidade de testbench sem portas (padrao do projeto)
13 -- ------------------------------------------------------------------------------
14 entity Controlo_Acessos_tb is
15 end entity Controlo_Acessos_tb;
16
17 architecture sim of Controlo_Acessos_tb is
18
19     -- Sinais ligados a UUT
20     signal A : std_logic_vector(3 downto 0);
21     signal S : std_logic_vector(1 downto 0);
22     signal R : std_logic;
23
24     -- 20 ns por estimulo (REQ-CTB-003)
25     constant TEMPO_ESTIMULO : time := 20 ns;
26
27 begin
28
29     -- ------------------------------------------------------------------------------
30     -- Instanciacao do modulo principal
31     -- ------------------------------------------------------------------------------
32     UUT : entity work.Controlo_Acessos
33         port map (
34             A => A,
35             S => S,
36             R => R
37         );
38
39     -- ------------------------------------------------------------------------------
40     -- Processo de estimulos: 5 cenarios reais de acesso
41     -- ------------------------------------------------------------------------------
42     stim_proc : process
43         variable erros : natural := 0;
44
45         -- Compara R com o valor esperado e regista divergencias
46         procedure verificar (esperado : std_logic; descricao : string) is
47         begin
48             if R /= esperado then
49                 erros := erros + 1;
50                 report descricao & " | ERRO: R=" & std_logic'image(R) &
51                        ", esperado=" & std_logic'image(esperado)
52                     severity error;
53             else
54                 report descricao & " | OK: R=" & std_logic'image(R);
55             end if;
56         end procedure;
57
58     begin
59         -- Cenario 1: funcionario 0 (permissoes: apenas D0) no departamento 0
60         -- Esperado: R = 1 (cancela abre, acesso autorizado)
61         A <= "0000";
62         S <= "00";
63         wait for TEMPO_ESTIMULO;
64         verificar('1', "Cenario 1 | Func 0 | Dept 0 | esperado R=1 (D0 autorizado)");
65
66         -- Cenario 2: funcionario 0 (sem permissao em D1) no departamento 1
67         -- Esperado: R = 0 (cancela fecha, acesso negado)
68         A <= "0000";
69         S <= "01";
70         wait for TEMPO_ESTIMULO;
71         verificar('0', "Cenario 2 | Func 0 | Dept 1 | esperado R=0 (D1 nao autorizado)");
72
73         -- Cenario 3: funcionario 4 (permissoes: D1 e D2) no departamento 1
74         -- Esperado: R = 1 (cancela abre, acesso autorizado)
75         A <= "0100";
76         S <= "01";
77         wait for TEMPO_ESTIMULO;
78         verificar('1', "Cenario 3 | Func 4 | Dept 1 | esperado R=1 (D1 autorizado)");
79
80         -- Cenario 4: funcionario 4 (permissoes: D1 e D2) no departamento 2
81         -- Esperado: R = 1 (cancela abre, acesso autorizado)
82         A <= "0100";
83         S <= "10";
84         wait for TEMPO_ESTIMULO;
85         verificar('1', "Cenario 4 | Func 4 | Dept 2 | esperado R=1 (D2 autorizado)");
86
87         -- Cenario 5: funcionario 8 (permissoes: apenas D3) no departamento 3
88         -- Esperado: R = 1 (cancela abre, acesso autorizado)
89         A <= "1000";
90         S <= "11";
91         wait for TEMPO_ESTIMULO;
92         verificar('1', "Cenario 5 | Func 8 | Dept 3 | esperado R=1 (D3 autorizado)");
93
94         -- Resumo final (auto-verificacao, REQ-CTB-004)
```

```
95          if erros = 0 then
96              report "=== SIMULACAO OK: 5/5 cenarios corretos, 0 erros ===";
97          else
98              report "=== SIMULACAO FALHOU: " & natural'image(erros) &
99                     " erro(s) em 5 cenarios ===" severity failure;
100         end if;
101
102         wait;  -- termina a simulacao (run -all)
103     end process stim_proc;
104
105 end architecture sim;
```

B Atribuição de Pinos

Tabela 3: Atribuição de pinos do módulo `Controlo_Acessos` na placa Intel DE10-Lite [2].

Sinal	**Periférico**	**Pino**
A_3	SW3	C12
A_2	SW2	D12
A_1	SW1	C11
A_0	SW0	C10
S_1	SW5	B12
S_0	SW4	A12
R	LEDR0	A8